Sensor-Aware Random Forests With Early Exits For Energy-Efficient Time Series Applications on IoT Devices

Anonymous submission

Abstract

Energy efficiency is a critical bottleneck for continuous time-series classification on edge IoT devices, especially in wearable health monitoring where battery constraints are severe. To address this, we introduce Sensor-aware Early-Exit Random Forests (SEE-RF) – the first architecture to incorporate early-exit mechanisms into Random Forests for joint sensing and inference optimization. By dynamically stopping the inference process

once sufficient classification confidence is reached, SEE-RF significantly reduces both sensing and computational energy. We evaluate SEE-RF using two variants – a vertically-tiered forest for full-storage settings and a cascade of sub-forests for ultra-low-memory devices—within a health monitoring context chosen for its stringent energy demands and vital societal impact. Across six diverse health datasets, SEE-RF achieves up to X % (X: 60 to 75, please check plots) sensing energy savings with negligible loss in accuracy. Our results demonstrate its potential to extend wearable device battery life and promote equitable access to continuous health monitoring. Additionally, we compare exhaustive search and Bayesian optimization for design-space exploration of SEE-RFs, and make our code and models publicly available, supporting sustainable and reproducible edge AI research.

1. Introduction

The deployment of edge artificial intelligence (AI) across billions of Internet of Things (IoT) devices has created an unprecedented energy crisis that threatens both the sustainability and accessibility of AI-powered technologies. Edge AI encompasses all forms of on-device machine learning inference, from neural networks to tree ensembles, executing directly on resource-constrained hardware without cloud connectivity. While data center AI energy consumption attracts significant attention [**?**], the distributed energy footprint of edge inference – occurring continuously on battery-powered wearables, environmental sensors, and embedded systems worldwide – represents an equally critical challenge [**?**]. With more than 1.1 billion wearable devices shipped globally in 2024, and projections reaching 1.8 billion by 2028 [**?**], the cumulative environmental and social impact of edge AI efficiency cannot be overstated.

The edge-AI energy crisis manifests across multiple applications, from smart agriculture sensors monitoring crop health to industrial IoT predicting equipment failures. However, it becomes most acute and consequential in wearable health monitors that run continuous time-series inference on batteries measured in milliamp-hours. For example, for the 422 million people with diabetes requiring continuous glucose monitoring, or the 1.3 billion with hypertension needing regular tracking, battery life determines whether AI-powered health monitoring is practical or aspirational. These devices represent an ideal case study for our approach: they combine strict power constraints with critical functionality that directly impacts human welfare.

Modern wearable devices execute multi-modal sensor fusion for heart-rate variability analysis, atrial fibrillation detection, and stress monitoring, yet battery technology advances at only 3-5% annually [**?**] while computational demands grow exponentially. Because current systems keep every sensor active until complete data windows arrive, sensing power dominates the energy budget [**?**], forcing frequent recharges. This widening performance-power gap forces users to choose between comprehensive health monitoring and acceptable battery life. current devices requiring daily charging in resource-constrained environments, effectively exclude entire populations from wearables.

Current time-series classification systems exhibit a fundamental inefficiency: they consume complete sensor windows and execute full inference pipelines regardless of classification difficulty. Consider activity recognition – distinguishing "sitting" from "running" often requires only the first 20% of data, yet conventional models waste 80% of sensing energy and computational cycles collecting redundant information. This "data-oblivious" approach directly conflicts with the resource constraints of battery-powered edge devices and represents a massive source of preventable energy waste.

To overcome the above challenges of energy inefficiency due to sensor data, we propose early exit random forest classifiers that make decisions with partial data to save sensing energy. Key idea in the proposed approach is to periodically generate application inference with partial sensor data and assess confidence of predictions. If the confidence of predictions exceeds a pre-defined threshold, the sensors are transitioned to a low-power mode until the next time-series window. This can lead to significant savings in energy if inference is made with partial data windows. The proposed approach significantly improves on existing solutions

that address only partial aspects of this problem. Dynamic CNNs like BranchyNet [?] and MSDNet [?] skip computation but still collect the entire window; adaptive sampling heuristics [? ?] throttle sensors but ignore inference cost. Recent work has explored early-stopping for Random Forests [?], but focuses solely on computational savings. Our work uniquely combines sensor control with early-exit inference, allowing both sensing and computation to adapt based on real-time prediction confidence.

We choose Random Forests as our foundation because they offer unique advantages for resource-constrained deployment: no specialized hardware requirements, predictable memory footprints, and inherent interpretability crucial for health applications. Unlike neural networks, tree ensembles execute efficiently on the simple microcontrollers found in affordable wearables. Their natural hierarchical structure – where tree depth encodes prediction difficulty – aligns perfectly with confidence-based early exits. We investigate two complementary architectures. SEE-RF maintains unified voting across depth-aligned tiers, ideal when memory permits storing the full forest structure. Sequential SEE-RF employs cascaded sub-forests with aggressive memory optimization, enabling deployment on ultra-constrained devices. This dual approach ensures our energy-saving techniques can benefit users across the economic spectrum.

We validate the proposed early exit RF classifiers with six health datasets ranging from activity recognition to electroencephalogram (EEG) processing. Evaluations with these datasets show that the proposed RF models achieve up to XX energy savings while maintaining accuracy levels similar to baseline RFs that use full data windows. Moreover, comparisons to prior approaches that perform early time series classification show that the proposed approach outperforms them in energy efficiency, thus showing the benefits of the proposed approach.

In summary, this paper makes the following key contributions:

1. Two versions of early-exit RF architectures that enable progressive inference in tree ensembles, addressing a critical gap in energy-efficient ML.
2. Unified framework that powers down sensors when predictions reach sufficient confidence, uniquely targeting both major energy consumers in edge devices.
3. Systematic optimization methodology combining design space exploration with Bayesian optimization, providing reproducible configuration for diverse applications.

Related Work

Early-exit architectures for neural networks have demonstrated significant computational savings in resource-constrained environments. BranchyNet [?] introduced intermediate classifiers at multiple network depths, enabling inference termination when prediction confidence exceeds learned thresholds. MSDNet [?] extended this paradigm through multi-scale feature processing for anytime prediction under computational budgets. These architectures achieve up to 5× reduction in computational cost while maintaining competitive accuracy. However, a fundamental limitation persists: both approaches require complete sensor data acquisition before inference begins. In edge devices where sensing power constitutes 60-80% of total energy consumption [?], computational optimization alone provides limited system-level benefits.

Complementary research has addressed sensor energy through early time-series classification. Xing et al. [?] pioneered algorithms that classify time series upon observing sufficient discriminative subsequences, demonstrating 70% reduction in data requirements across multiple domains. Martínez et al. [?] formulated early classification as a reinforcement learning problem, learning optimal stopping policies that outperform hand-crafted heuristics. TEASER [?] represents the current state-of-the-art, employing a two-tier architecture where slave classifiers determine safe stopping points and a master classifier provides final predictions. Despite their sophistication, these methods exhibit three critical limitations for edge deployment: (i) they operate on pre-collected datasets rather than real-time sensor streams, (ii) stopping decisions remain decoupled from inference model confidence, and (iii) they provide no mechanism for computational adaptation based on classification difficulty.

Recent work by Daghero et al. [?] demonstrates early stopping for Random Forests on RISC-V microcontrollers, achieving up to 90% reduction in computational energy. Their approach monitors cumulative class scores and terminates execution when margins exceed a threshold. While validating that Random Forests can benefit from early stopping, their solution addresses only computation: sensors remain active throughout windows, the forest architecture stays unmodified, and a single global threshold cannot adapt to varying exit depths. This implementation-level approach misses the fundamental opportunity to redesign Random Forest architectures for joint sensor-compute optimization.

In strong contrast to prior approaches, our work addresses this gap through architectures specifically designed for progressive inference with integrated sensor control capabilities, enabling synergistic energy savings that isolated optimizations cannot achieve.

Background and Problem Setup

Time-Series Classification on Edge Devices

Edge-based time-series classification underpins numerous real-world applications, from continuous health monitoring to industrial predictive maintenance. These systems operate through a well-defined pipeline: sensors sample physical phenomena at fixed rates, accumulate measurements into temporal windows, and apply machine learning models to produce classifications.

The energy profile of edge inference comprises two primary components: *sensing energy* E_{sense} consumed during data acquisition and *computational energy* $E_{compute}$ required for model execution. Empirical studies demonstrate that sensing power often dominates total system consumption [?]. Sensors typically remain active throughout the entire window duration T, consuming energy proportional to $P_{sensor} \times T$ regardless of the intrinsic difficulty of the classification task. This *data-oblivious* approach creates substan-

tial energy waste. Current systems consume complete sensor windows and execute full inference pipelines regardless of classification difficulty.

Random Forests for Edge Deployment

Random Forests present several architectural advantages that make them particularly suitable for resource-constrained edge environments. As ensemble methods, Random Forests aggregate predictions from multiple decision trees, each trained on bootstrapped data samples []. The prediction process involves traversing each tree from root to leaf based on feature comparisons, followed by majority voting across the ensemble. This architecture offers four key benefits for edge deployment:

Computational Simplicity: Tree traversal requires only comparison operations at each node, avoiding the computational complexity of neural networks. The worst-case inference complexity is $O(N \times D)$ where N denotes the number of trees and D represents maximum tree depth [].

Memory Determinism: Unlike neural architectures that require dynamic memory allocation for intermediate activations, Random Forests maintain static memory footprints.

Hardware Agnosticism: Random Forests execute efficiently on general-purpose processors without specialized accelerators. Tree-based models require no specialized hardware accelerators and provide inherent interpretability.

Architectural Design Choices for Early Exit RFs

Early-exit architectures must balance several competing objectives: prediction confidence, memory footprint, and deployment flexibility. We investigate two complementary approaches to address different deployment scenarios:

- **Unified voting across depths (SEE-RF)**: Maintains the standard Random Forest voting mechanism while enabling early termination at synchronized tree depths. This preserves ensemble diversity and is ideal when memory permits storing the full forest structure.
- **Progressive cascade refinement (Sequential SEE-RF)**: Employs smaller sub-forests in sequence, each refining the previous prediction. This approach minimizes memory footprint through node sharing and suits ultra-constrained devices where memory is limited.

Problem Formulation

We formalize the sensor-aware early-exit problem as follows. Consider a time-series classification task with window duration T, sampling frequency f_s, and C target classes. We define K potential exit points at times $\mathcal{T} = \{t_1, t_2, ..., t_K\}$ where $0 < t_1 < t_2 < ... < t_K = T$. At each exit point $t_i \in \mathcal{T}$, the system observes sensor data $\mathbf{x}_{1:t_i}$ and must decide between two actions:

1. **Accept**: Terminate inference with prediction $\hat{y}_i = f_\theta(\mathbf{x}_{1:t_i})$ and transition sensors to low-power mode
2. **Continue**: Proceed to next exit point t_{i+1}, collecting additional data $\mathbf{x}_{t_i+1:t_{i+1}}$

The total energy consumption for a window with early exit at time t_i equals:

$$E_{total}(t_i) = \underbrace{P_{sens} \times t_i}_{E_{sens}(t_i)} + \underbrace{\sum_{j=1}^{i} E_m(t_j)}_{E_{compute}(t_i)} + \underbrace{E_{sleep} + E_{wake}}_{E_{transition}}$$

where $E_m(t_j)$ represents the computational energy for evaluating tier j, and $E_{transition}$ captures sensor state-change overhead. Our optimization objective minimizes expected energy consumption while maintaining classification performance:

$$\min_{\theta, \boldsymbol{\tau}} \mathbb{E}_{\mathbf{x}, y \sim \mathcal{D}}[E_{total}(\mathbf{x})] \quad \text{s. t.} \quad \text{Acc}(\theta, \boldsymbol{\tau}) \geq \text{Acc}_{baseline} - \epsilon$$

where θ denotes model parameters, $\boldsymbol{\tau} = [\tau_1, ..., \tau_K]$ represents exit thresholds, $\mathcal{D}$ is the data distribution, and ϵ specifies acceptable accuracy degradation (typically 0.01 or 1%).

Confidence Quantification via Entropy

The decision to accept or continue relies on quantifying prediction confidence. We employ normalized Shannon entropy as our confidence metric:

$$\phi(\mathbf{p}) = -\frac{1}{\log C} \sum_{c=1}^{C} p_c \log p_c \tag{1}$$

where $\mathbf{p} = [p_1, ..., p_C]$ represents the predicted class probability distribution. The normalization factor $\frac{1}{\log C}$ ensures $\phi(\mathbf{p}) \in [0, 1]$ regardless of the number of classes.

Shannon entropy provides several advantages for early-exit decisions:

- **Uncertainty Quantification**: Entropy naturally captures prediction uncertainty—low entropy indicates confident predictions while high entropy signals ambiguity
- **Computational Efficiency**: Computing entropy requires $O(C)$ operations, negligible compared to inference
- **Calibration**: Entropy correlates well with actual prediction accuracy, providing reliable confidence estimates (validated empirically in Section **??**)

The system accepts a prediction when $\phi(\mathbf{p}) < \tau_i$, where τ_i denotes the learned confidence threshold for exit i. Lower thresholds require higher confidence, trading energy savings for accuracy.

Proposed Approach

In this section we describe our proposed models that target energy consumption minimization in time-series applications. Our solution incorporates the usage of random forest with early exits where we implement two different versions of early exit RFs. The first RF framework includes vertical exits, and the second framework uses a sequence of RFs to enable horizontal early exits. A unified high-level workflow for both approaches is illustrated in Figure 1. Details of each framework are presented in the following sections.

The time series training data we are using consists of multi-modal temporal measurements:

$$\mathcal{X} \in \mathbb{R}^{n_{\text{windows}} \times n_{\text{points}} \times n_{\text{channels}}}$$

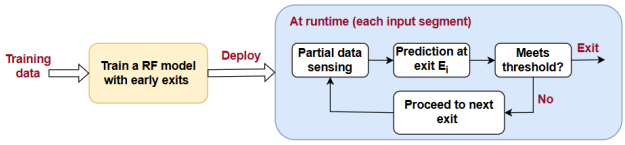

Figure 1: SEE RF-V and SEE SRF-H workflow

Where n_{windows} is the number of extracted windows, n_{points} is the number of samples per window, and n_{channels} is the number of sensors.

SEE RF-V: Sensor Aware Early Exits Tiered Vertical Random Forest

The SEE RF-V architecture structures each decision tree in the RF into multiple depth-aligned tiers with exits enabling the model to halt inference early whenever a confidence threshold is met, as shown in Figure 2. Nodes splitting in every each decision tree follows the standard information gain maximization but with a depth budget at every each tree portion. The depth budget ensures that XX. Moreover, in each tier, we constraint the features space to a specific features subset to ensure that partial window data are used in the early exits. For instance, the first tree split may only use 30% of the window data to determine the class predictions.

We keep expanding the trees in a progressive tier-wise manner while incrementally introducing new early exit points at the end of each tree portion. This process continues until either the maximum tree depth is reached, the number of samples at a node falls below the predefined minimum threshold, or the node achieves class purity []. The overall process of training the random forest is described in Algorithm 1 where we show the high level steps of the SEE RF-V model training.

During inference, the SEE RF with vertical exits operates as follows: the sensors start reading a specific data portion from the data window. This data portion will be evaluated using the first tree portion. At the first exit, a decision is made based on the entropy of the predictions. If the entropy is less than the confidence threshold, that means that the model is deemed confident enough to exit early and sensors are transitioned into a low power mode.

Conversely, if the entropy exceeds confidence threshold, it indicates that more data should be added to produce a more informed decision. In this latter case, we add a new data portion to the previous one, and move to the next exit. This process is repeated until an early exit is taken or we reach the terminal exit.

SEE SRF-H: Sensor Aware Early Exits Sequential RF with horizontal Exits (Cascaded Sub-Forests)

The Cascaded Sequential Random Forests with Early the Exit (SEE SRF-H) framework is designed as a single model that extends the concept of sequential Random Forest (sRF) by organizing multiple sRF instances into a cascaded architecture.

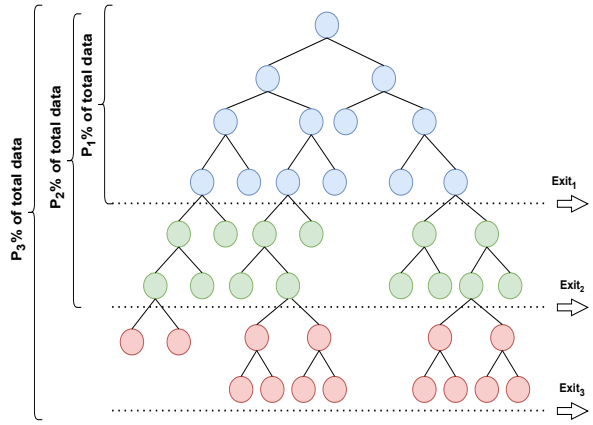

Figure 2: Tree architecture in SEE RF-V

The Sequential Random Forest (sRF) introduced by Baumann et al[1], have shown improved detection accuracy for RFs. This improvement is achieved through a sequential adaptation of training samples and a weighted majority voting scheme. Inspired by boosting methods, this approach enables the decision trees within a single forest to complement each other more effectively. However, while the sRF enhances accuracy, it processes all samples through the entire one forest, which may not be ideal for energy-constrained applications where computational efficiency is a primary concern. Our approach leverages sRF and addresses the challenge of energy-efficient sensor data classification by structuring the classification process into a series of interconnected sub-forests. The following sections describe the details of the sRF with early exits.

Cascaded Architecture and Inter-Forest Weight Transfer (Training)

This section describes the process to train the sRFs with early exit. As shown in Figure 3, the forest is divided into multiple sub-forests, each corresponding to an early exit. For instance, the first sub-forest includes a subset of trees that are used with the first data portion. Subsequently, the next sub-forest is invoked if the predictions from the first exit do not have a high confidence. The core idea of each sub-forest is to sequentially adapt the number of training samples per class for each tree and to employ a weighted majority voting. Each sub-forest is trained using the following process:

1. **Sample Re-weighting**:

   In sRF Decision trees are not grown independently. Instead, the training process for each tree is influenced by the performance of the previously trained trees within the same sub-forest. This influence is achieved by introducing sample weights, which are initially uniform across all training samples.

$$w^0(i) = \frac{1}{N}, \quad i = 1, 2, ..., N$$

Algorithm 1: Training SEE RF-V

Require: Dataset D, pre-processed feature partitions $Data_P$, depth split ratios $Splits$, number of estimators $N_{\text{estimators}}$, maximum tree depth max_depth

Ensure: List of trained trees with vertical early-exit structure

```
window_length ← length of input vector D[0]
Trees_list ← [ ]
for i = 1 to N_estimators do
    D_i ← Bootstrap sample from D with replacement
    current_node ← None
    for j = 1 to length(Splits) do
        if current_node = None then
            T_i ← Build tree using only Data_P[j] of window_length up to depth Splits[j] × max_depth
        else
            T_i ← Extend T_i from current_node using only Data_P[j] of window_length up to depth Splits[j] × max_depth
            current_node ← Store frontier nodes at current depth as exit points
    Append T_i to Trees_list
return Trees_list
```

where N is the total number of training samples. For each subsequent tree t, training samples are chosen based on updated class-specific weights, $W^t(c)$, which reflect the performance of the preceding trees on out-of-bag (OOB) samples. OOB samples are those from the original training set that were not included in the bootstrap sample used to train a particular tree, effectively serving as an internal, unbiased test set. After each tree t is constructed within a sub-forest, its performance is evaluated on its OOB samples. The weights of correctly classified samples remained, while samples that were misclassified by previous trees increased their weights, ensuring that the current tree focuses on "harder" examples.

2. **Adaptive Sample Selection**:

   Adjusted weights allow adaptive sample selection for each class when learning the next tree, enabling trees to better complement the existing sub-forest ensemble.

3. **Weighted Majority Voting**:

   For final classification within a sub-forest, a modified majority voting is used. Unlike standard RFs where each tree has an equal impact, the influence of each tree's prediction is modulated by its class-specific error rate, which is computed using the OOB samples. This approach reduces the impact of poorly performing trees for specific classes and amplifies the influence of highly accurate ones, contributing to the overall improved accuracy of the sRF.(Should I mention the formula from that paper too?)*******

   This internal sRF mechanism allows each sub-forest to achieve high accuracy by building a strong, complementary ensemble of trees. A fter this process is finished with the first sub-forest, we pass the wights of last tree as a initial weight for the next sub-forest's first tree. And repeat this process for each sub-forest. Instead of treating each sub-forest independently, we propagate information from the preceding sub-forest to initialize the subsequent one. Specifically, the classification confidence or learned sample distribution from the final tree of sub-forest SFk is used to inform the initial weighting of samples for the first tree of sub-forest SFk+1.Therefor at the end of training, we have multiple sub-forests which are connected to each other and inside each sub-forest, we have a mechanism that improves the model.

Inference Phase:

To achieve energy efficiency, the model uses an entropy-based early exit strategy. During inference, after a sample x has been processed by a sub-forest SF_k, its classification confidence is evaluated using the entropy of its predicted class probabilities. Lower entropy indicates higher confidence in the classification. For each sub-forest (SF_k), if the entropy of a sample is below a certain threshold, the sample (x) is considered confidently classified by SF_k and exits the pipeline. These samples contribute to the final classification result. Samples with entropy greater than the threshold are classified as uncertain and forwarded to the next sub-forest (SF_{k+1}) for further processing. The final sub-forest (SF_L) does not allow for an early exit; all samples that reach it are classified by SF_L.

This early exit strategy is essential for reducing energy consumption. By classifying and exiting confident samples early, the subsequent sub-forests work with a significantly smaller dataset. This leads to a decrease in computational cycles, memory access, and overall energy usage. The thresholds can be adjusted to find the right balance between accuracy and energy efficiency.

Entropy-based Confidence Thresholds

Confidence metric. At every early-exit tier we compute *normalized Shannon entropy*. Entropy is (i) parameter-free and bounded, which simplifies threshold search; (ii) monotonically related to vote margin, providing an intuitive uncertainty ranking; and (iii) widely adopted in prior dynamic-inference work, enabling direct comparison. Using this metric both SEE-RF variants (****and the reference SEEN-CNN******) share a single exit criterion. Measured on our test sets, each exit remains well calibrated with ECE ≤ 0.05 (results from pareto optimal configs).

Formally, for a C-class problem the entropy of a sample at tier t in vertical tiered SEE-RF or at a sub-RF t in sequential SEE-RF is:

$$H_t = -\sum_{c=1}^{C} p_t(c)\ \log\big(p_t(c) + \epsilon\big),$$

where $p_t(c)$ is the fraction of trees (at tier t in vertical tiered SEE-RF or at sub-RF t in Sequential SEE-RF) voting for class c. We use $\epsilon = 10^{-5}$ in order to avoid

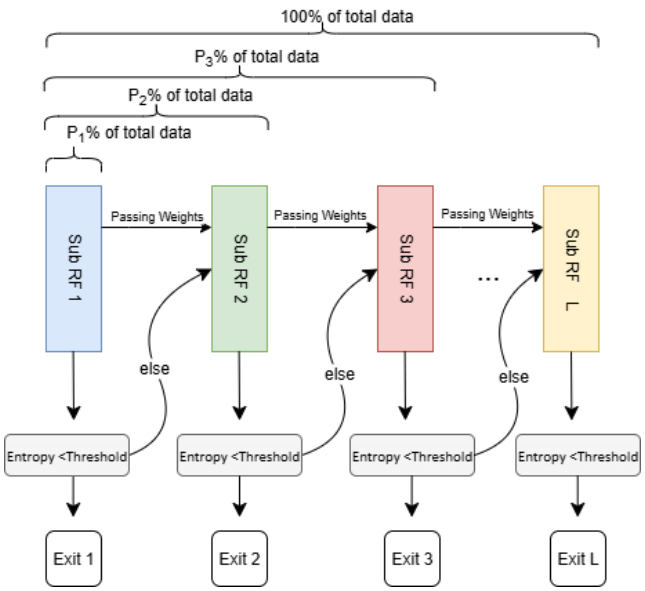

Figure 3: Tree architecture in RF with horizontal exits

taking the logarithm of zero.

Threshold selection. Building on insights from BranchyNet, we evaluated three threshold ordering strategies. To set practical thresholds, we partition the interval $[0, H_{\max}]$ into B equal-width bins (with $B = 4$), where $H_{\max}$ denotes the maximum entropy corresponding to a uniform class distribution.
Although finer or adaptive binning could be explored, this simple random selection scheme already yields strong energy–accuracy trade-offs and low ECE; thus further threshold optimization is left to future work.

We explored three assignment strategies:

- **Strictly increasing (ascending)**: Enforces progressively stricter thresholds at deeper exits. It might limit early exits as most samples only meet confidence requirements at final tiers, which may decrease energy savings achieved by the model.
- **Decreasing**: Relaxes thresholds at deeper exits and maximizes early exits, and it might led to better energy savings (in compare to ascending).
- **Random assignment**: Independently selects thresholds from the candidate bins for each exit without ordering constraints.

******** this paragraph needs to be modified according to our findings*******
Through empirical evaluation, we found that random threshold assignment achieves the best accuracy-energy trade-off (X% accuracy with Y% energy savings, as it allows each exit to adapt to its natural confidence characteristics without imposing global constraints. This counterintuitive finding suggests that rigid threshold schedules may be overly restrictive for tree-based ensembles.

Algorithm 2: Training SEE SRF-H with Weight Propagation and Feature Partitioning

Require: Dataset D, Number of sub-forests L, Preprocessed feature partitions $Data_P$, Number of estimators per sub-forest $N_{\text{estimators}}$, Maximum tree depth max_depth.
Ensure: List of trained sub-forests $[SF_1, \ldots, SF_L]$.
$SubForests \leftarrow [\,]$
Initialize sample weights $w(i) \leftarrow 1/N$ for all $i = 1, ..., N$.
for $l = 1$ to L **do**
  $Current_SF \leftarrow [\,]$
  for $t = 1$ to $N_{\text{estimators}}$ **do**
    Calculate class weights $W(c) = \sum_{i:y_i=c} w(i)$ for each class c.
    Create bootstrap sample D_t from the full dataset D based on weights W.
    Let $OOB_t = D \setminus D_t$ be the out-of-bag samples.
    Train a decision tree estimator h_t on D_t using features from partition $Data_P[l]$ up to depth max_depth.
    Compute class-specific error rates $\text{error}_t(c)$ using h_t on OOB_t.
    Update sample weights w by decreasing weights for correctly classified samples in OOB_t.
    Append (h_t, error_t) to $Current_SF$.
  Append $Current_SF$ to $SubForests$.
return $SubForests$

Design Space Exploration

Our design space exploration encompassed model architectures (**********?**********), hyperparameters, and threshold selection strategies. In an effort to identify the best design space exploration practice, we perform two different DSEs: exhaustive search with pareto optimal configuration extraction, and Bayesian optimization.

For exhaustive search, we evaluated a wide range of tree depths, number of estimators, data percentages, tree splits (for SEE RF-V), sub-RF splits (for SEE SRF-H) (*** we need to make it more clear****), and entropy thresholds. After that, we extracted the pareto optimal configurations from the exhaustive search results in order to get the best accuracy-energy trade-off. In order to reduce the cost of this exhaustive search while exploring diverse regions, we used a step size for each parameter and then performed a random parameter extraction.

For Bayesian optimization, we used the same search space as in exhaustive search. Unlike the exhaustive search, the Bayesian optimization did not use predetermined threshold orderings; instead, we defined the search space to allow independent threshold selection at each exit, enabling the optimizer to discover promising configurations without constraining the threshold relationships.

The dual-objective Bayesian optimization problem is

stated as follows:

$$\min_{\mathbf{x}\in\mathcal{X}} [-Acc(\mathbf{x}),\ E(\mathbf{x})] \tag{2}$$

where $Acc(\mathbf{x})$ is the model accuracy, $E(\mathbf{x})$ is the total energy consumption (including both sensor and compute energy), and $\mathcal{X}$ is the configuration space. The final outcome of this DSE are the pareto optimal configurations that offers the best accuracy-energy trade-off.

We evaluate and compare both design space exploration methods, with the corresponding results presented in the Experimental Results section.

Experimental Results

Platform and Evaluation Context

********** Exhaustivve and BO search have not been performed on device; so we need to push this paragraph after the results we got using exhaustive and BO **********

*******(We need to talk about evaluation metrics)*****

We evaluate our methods on the NVIDIA Jetson Xavier AGX in 10W low-power mode—a platform that has become a de facto standard for edge AI research [**?**] and represents realistic power constraints for next-generation wearable and IoT deployments. The Xavier provides a fair wearable proxy through its integrated INA226 power monitoring rails and established use in the edge AI research community for comparable evaluations. The on-chip power monitoring capabilities enable precise energy measurements that capture both sensing and computational components, providing the ground truth necessary for meaningful energy-efficiency claims in real-world deployment scenarios.

Our dataset selection emphasizes health and activity monitoring applications, which provide ideal evaluation conditions due to their well-characterized sensor modalities, established accuracy benchmarks from clinical studies, and clear real-world deployment constraints where energy efficiency directly impacts user welfare. Representative industrial time-series tasks (AI4I Predictive Maintenance, Occupancy Detection, Electricity Load Consumption) demonstrate broader applicability across IoT domains, confirming that our entropy-based early-exit framework generalizes beyond health-specific applications to realistic operational environments.

Exhaustive search results:

Here we plot the exhaustive search results before the pareto optimal configuration extraction for both models in order to show the overall performance of the model.

Accuracy vs tree depth: (check plots in appendix)

We start the experimental results with a description of accuracy with tree deoth. The plots show the accuracy of recognition with the depth of respective trees. Each bar represents a dataset that we used in the experiments.

We see that all datasets achieve high accuracy with SEE

- **SEE RF-V;** SEE RF-V performed well on Shoaib, PAMAP2 and SR-SCP1 where the accuracy was consistently higher than the baseline in SR-SCP1, equal to the baseline in PAMAP2, and at most 5% less than the baseline in Shoaib. The tree depth did not have any significant influence on accuracy in these 3 datasets. For Epilepsy and EMG, the accuracy was between 10%-15% less than the baseline accuracy. SEE RF-V leveraged the worse performance on WESAD in terms of accuracy; it went beyond 15% less than the baseline for some configurations with number of exits equal to 3 or 4. In these last three datasets, for a fixed number of exits, the accuracy changes in function of tree depth and it depends and the pattern it is dataset dependant. For instance, in WESAD, for number of exits equal to 3, the best accuracies were achieved for depths equal to 80 and 90; shallower depths or larger depths gave bad accuracy; while in Epilepsy, for the same number of exits equal to 3, the best accuracies were achieved by shallower depths and larger depths.
- **SEE SRF-H:** SEE SRF-H had similar performance to RF-H on PAMAP2 where the accuracy was equal to the baseline. In Shoaib, SEE SRF-H seemed to be slightly outperformed by SEE RF-V, and it slightly outperformed SEE RF-V in SR-SCP1. In addition, it had a significant better accuracy than SEE RF-V in WESAD and EMG, where the overall decrease in accuracy did not exceed 10%. In these two last datasets, for a fixed number of exits, the tree depth did not seem to significantly affect the accuracy, but the number of exits did; in fact less exits seemed to leverage the best accuracies. For Epilepsy, SEE SRF-H had approximately similar performance as SEE RF-V, and the influence of tree depth depends on the number of exits in the model. For instance, for a model with 3 or 4 exits, the accuracy decreased when moving from depth 50 to 80, but it improved when moving from 90 to 100.

SEE SRF-H shows stronger performance in compare to SEE RF-V on most of the datasets except for Shoaib. This advantage stems from the fact that horizontal exits aggregate decisions from full-depth trees, preserving the representational power of the ensemble. In contrast, vertical exits may force premature decisions when early exits occur at shallow depths, especially on complex datasets

Energy vs tree depth: (check plots in appendix)

- SEE SRF-H achieved significant energy savings, reaching close to 80% for Shoaib and PAMAP2 with 4 exits. For the other four datasets, models with 4 exits consistently delivered energy savings around 60%, and using more exits seemed to help increasing the energy saved. SEE RF-V also performed a significant energy savings for Shoaib and PAMAP2 though it consistently fell short of the savings achieved by SEE SRF-H.. For the other four datasets, the energy savings were below 60%. Adding more exits did not seem to improve energy savings. This is likely because, in SEE RF-V, exit opportunities are tied to vertical tree depth,

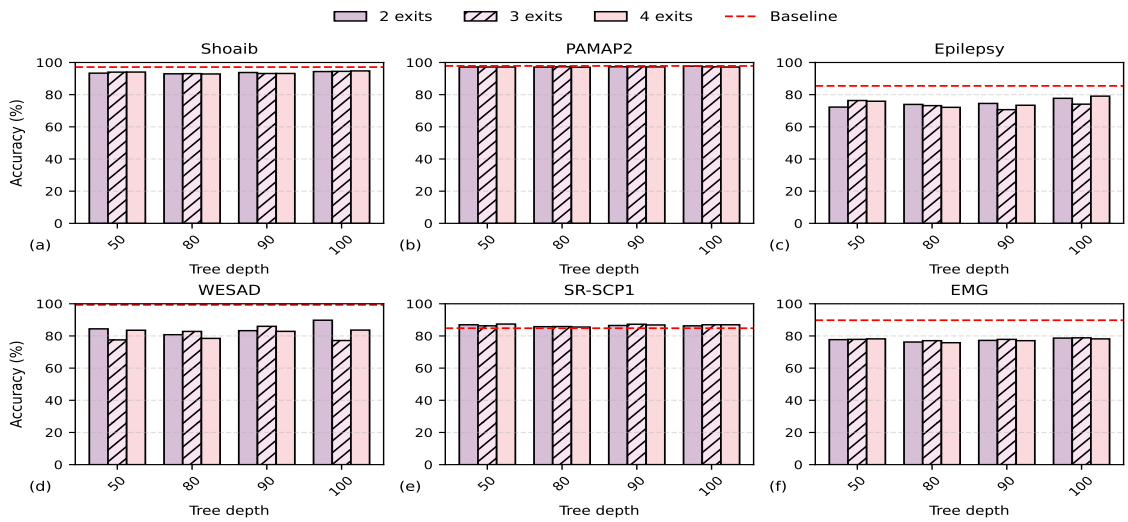

Figure 4: SEE RF-V: Accuracy vs. Tree Depth

which may not offer sufficient flexibility for early classification on more complex or noisy datasets.

Model ECE: (check plot in appendix)

- ECE values are generally lower in SEE SRF-H than RF-V across most dataset suggesting that SEE SRF-H is more calibrated than RF-V. This can be explained by the fact that every each layer can specialize and recalibrate predictions as more context is available, while SEE RF-V exits can be abrupt and may not see enough variance to produce calibrated probabilities. Nevertheless, SEE RF-V had lower ECE for WESAD and SR-SCP1 in compare to SEE SRF-H. This suggests that SEE RF-V is well calibrated on SR-SCP1 and WESAD (
******** we need more explanation**********
).

Comparing both DSEs methods:

In order to decide which best practice to adopt for design space exploration, we compare the results of both methods: exhaustive search (after pareto optimal configurations extraction), and bayesian optimization. Both methods yield the optimal configuration with the best trade-off between energy efficiency and accuracy.

Accuracy-Energy trade-off using pareto optimal points: (check plots in appendix)
By evaluating both DSEs methods, we can see that the points identified by DSE1 and DSE2 are closely clustered, which suggests that both methods converge to similar optimal solutions. Given that exhaustive search is time consuming, and that we were able to achieve similar results as exhaustive using a small number of trials (10 trials), we can say that bayesian optimization is the optimal design space exploration and should therefore be chosen as the preferred method.

Memory Footprint:

Here we compare the baseline model and our models memory footprints. We chose configurations from the Bayesian Optimization results.

	Memory Storage (KB)		
Dataset	Baseline RF	SEE RF-V	SEE SRF-H
Shoaib	2665.8	411	982
PAMAP2	942.0	100	96
Epilepsy	489.2	58.31	62
WESAD	113.1	237.19	5
SCP	395.2	39.7	144
EMG	879.2	253.92	223

Table 1: Summary of memory overhead of baseline and SEE-RF classifiers

We estimate the memory consumption of the model by assuming a fixed cost of 48 bytes per tree node, which aligns with standard Random Forest implementations. To calculate the total memory footprint, we multiply the number of nodes by the size per node and then convert the result to kilobytes. Since the architecture of models varies, it is evident that they perform differently in terms of memory usage.

Ablation study

We will picked two datasets: one with the best performance (SR-SCP1) in order to show where our models excel, and one with the lowest margin improvements (WESAD) in order to show where the models struggle or give modest gains(but how, we need to whow this later on).

******* for every each model we chose 2 datasets; we need to check if this is correct **********

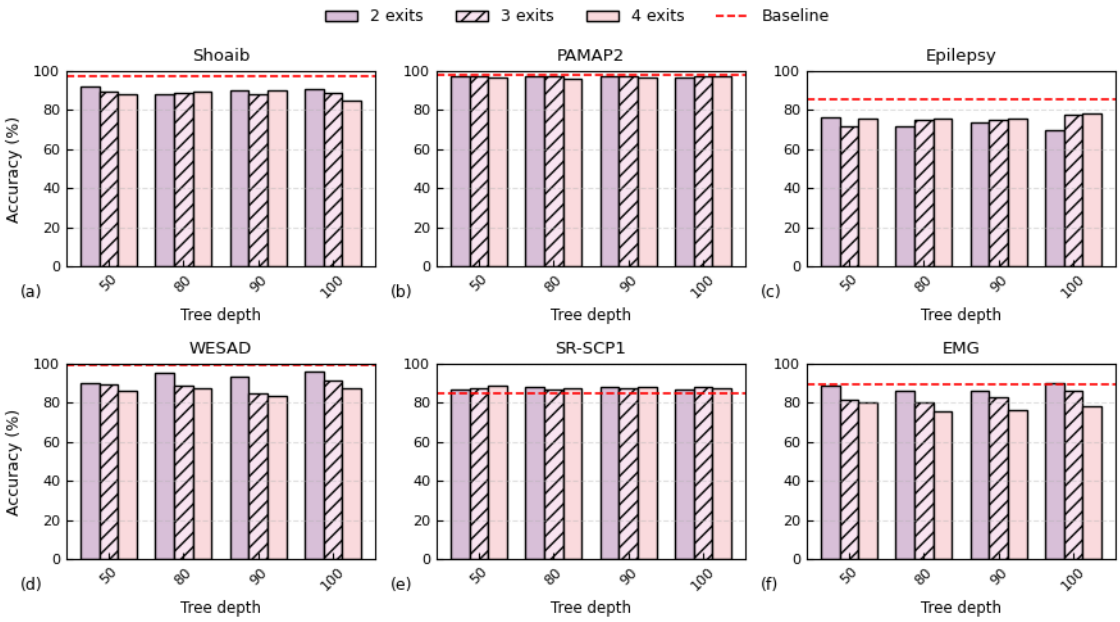

Figure 5: SEE SRF-H: Accuracy vs. Tree Depth

- **Importance of Exits:**

Model	SR-SCP1			WESAD		
	Acc%	E%	ECE	Acc%	E%	ECE
RF	90.27	0.0	0.09	100	0.0	0.0
RF-V	87.61	64.71	0.08	89.83	59.0	0.04

Table 2: Comparison of SEE RF-V against vanilla Random Forest (RF): accuracy, energy savings and ECE

Model	SR-SCP1			WESAD		
	Acc%	E%	ECE	Acc%	E%	ECE
SRF	84.7	0.0	0.09	100	0.0	0.0
SRF-H	90.2	75.0	0.09	76.4	75.0	0.08

Table 3: Comparison of SEE SRF-H against vanilla Random Forest (RF): accuracy, energy savings and ECE

Both tables show that the SEE RF-V and SEE SRF-H models significantly save energy compared to standard Random Forest models with a negligible decrease in accuracy (in the best performing dataset).The early-exit mechanism effectively reduces computational load by allowing models to stop inference once enough confidence is reached, without significantly affecting predictive performance. ECE is considered to be good here (I think it is moderate not good, we need to check).

- **Importance of sensor awareness:**

Model	SR-SCP1			WESAD		
	Acc%	E%	ECE	Acc%	E%	ECE
RF-V (100%)	85.84	0.0	0.06	100	0.0	0.0
RF-V (aware)	87.61	64.71	0.08	89.83	59.0	0.04

Table 4: Effect of sensor awareness: comparison of RF with vertical exits using 100% of data at each exit versus sensor-aware variant (accuracy, energy savings and ECE)

Model	SR-SCP1			WESAD		
	Acc%	E%	ECE	Acc%	E%	ECE
SRF-H (100%)	87.6	0.0	0.07	100	0.0	0.0
SRF-H (aware)	90.2	75.0	0.09	76.4	75.0	0.08

Table 5: Effect of sensor awareness: comparison of sequential RF with horizontal exits using 100% of data at each exit versus sensor-aware variant (accuracy, energy savings and ECE)

In the sensor-aware variant, we observe a good balance between energy savings and accuracy for both versions with an improving accuracy in the best performing dataset, and a negligible decrease in accuracy for the worse performing dataset. ECE is considered to be good here (I think it is moderate not good, we need to check).

- **Importance of threshold order protocol:**

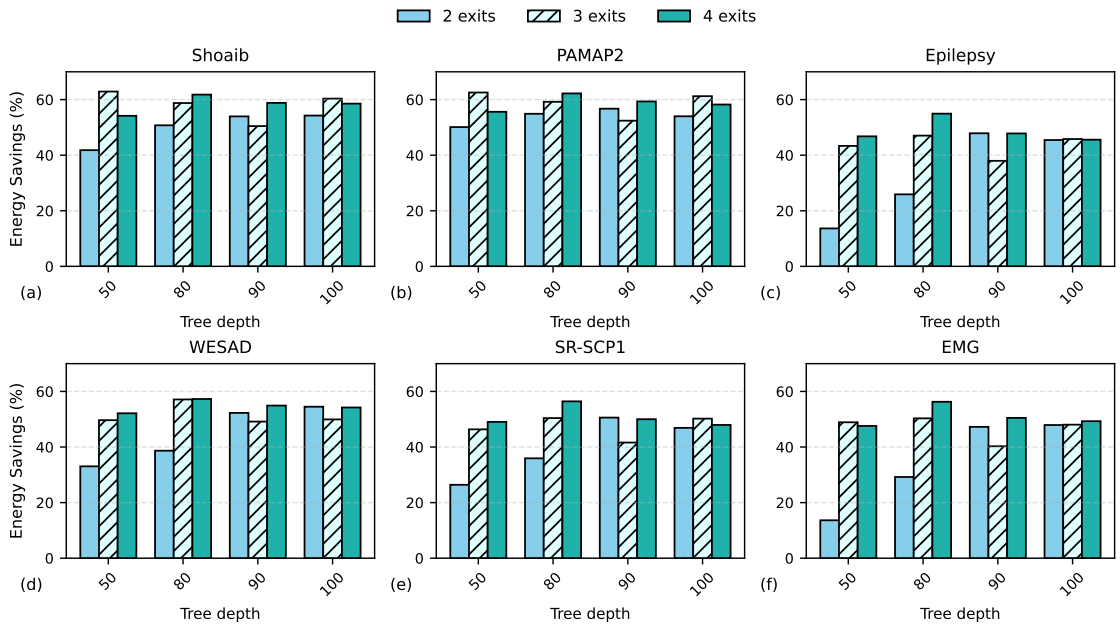

Figure 6: SEE RF-V: Energy vs. Tree Depth

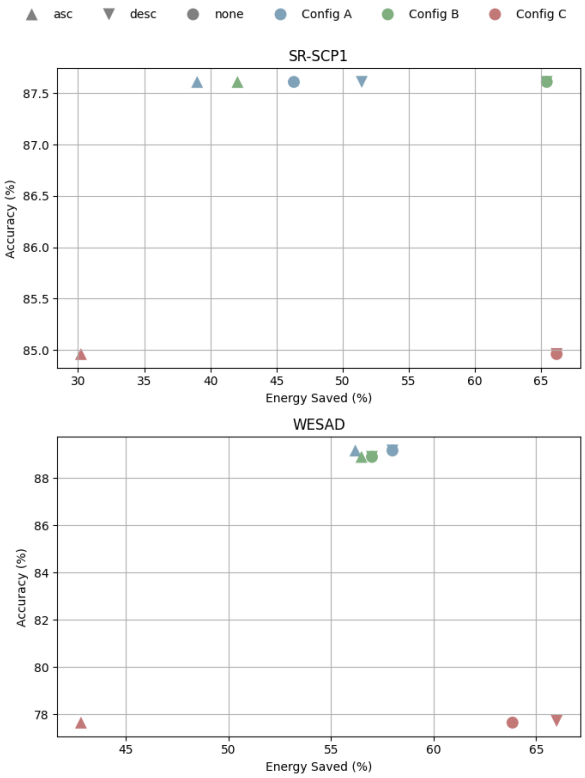

Figure 12: Effect of entropy-based thresholds ordering in SEE RF-V: comparison of ascendant, descendant, and unordered selection strategies.

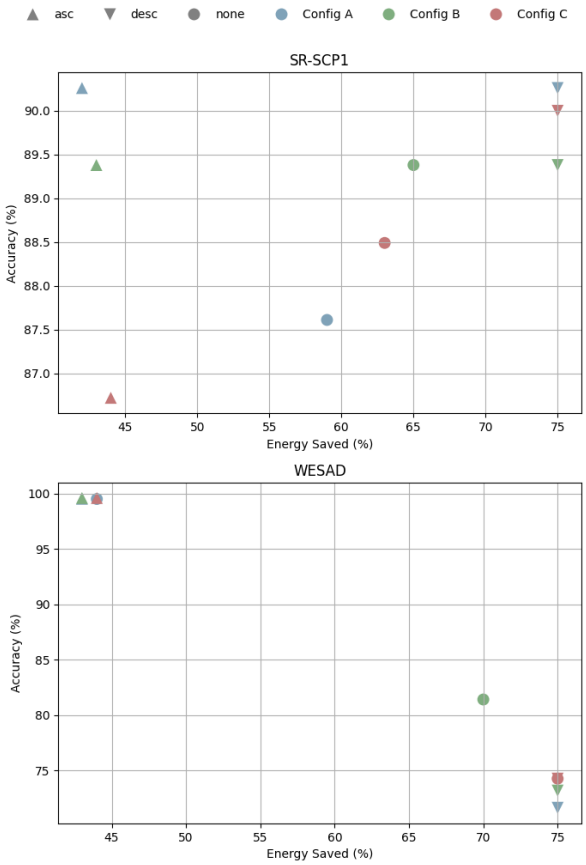

Figure 13: Effect of entropy-based thresholds ordering in SEE SRF-H: comparison of ascendant, descendant, and unordered selection strategies.

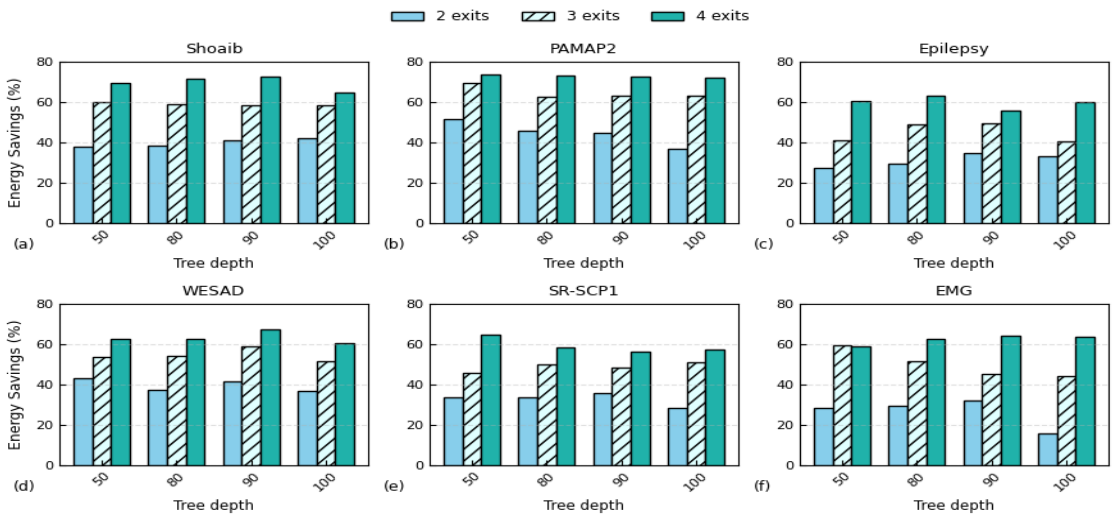

Figure 7: SEE SRF-H: Energy vs. Tree Depth

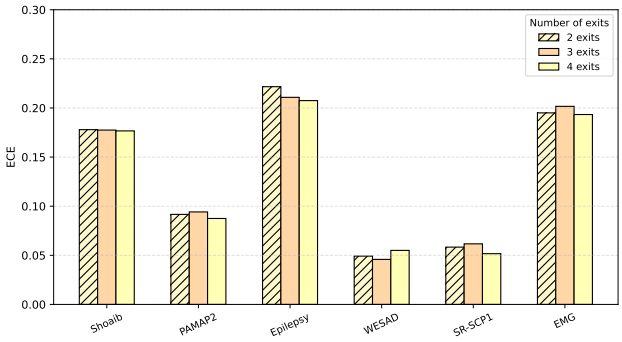

Figure 8: SEE RF-V; Model ECE

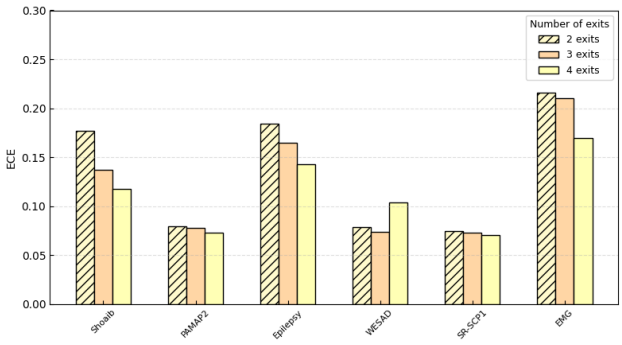

Figure 9: SEE SRF-H; Model ECE

Here, we chose 3 different configurations; for each configuration we fixed number of estimators, number of trees, thresholds list ... and we kept changing the threshold order; For each configuration, we varied only the threshold order — using ascending, descending, and unordered sequences — to evaluate its effect on performance.

For SEE RF-V model, all three thresholds ordering protocols in one single configuration leveraged the same accuracy, but different energy savings amounts; the pattern that was detected in most configurations was: ascending thresholds have the least amount of energy savings, unordered one have better, and descending have the highest energy savings.

For SEE SRF-H, for energy savings we can say that we have the same pattern detected in SEE RF-V.In contrast, accuracy was more sensitive to the threshold ordering protocol but no clear trend was observed. In both models, ECE stays the same while changing the thresholds order.

Deployment Architecture

Shorten it and give more details in the app. The energy efficiency gains from SEE-RF models can extend the operational lifetime of health monitoring devices, making continuous health tracking more accessible in resource-limited settings. This section describes how these models would be deployed to achieve practical energy savings through intelligent sensor management.

Signal Flow and Window Processing

The deployment pipeline would operate as follows:

(a) **Data Streaming**: Sensor continuously streams data to CPU at rate f_s

(b) **Window Assembly**: CPU collects samples into fixed-duration windows (non-overlapping)

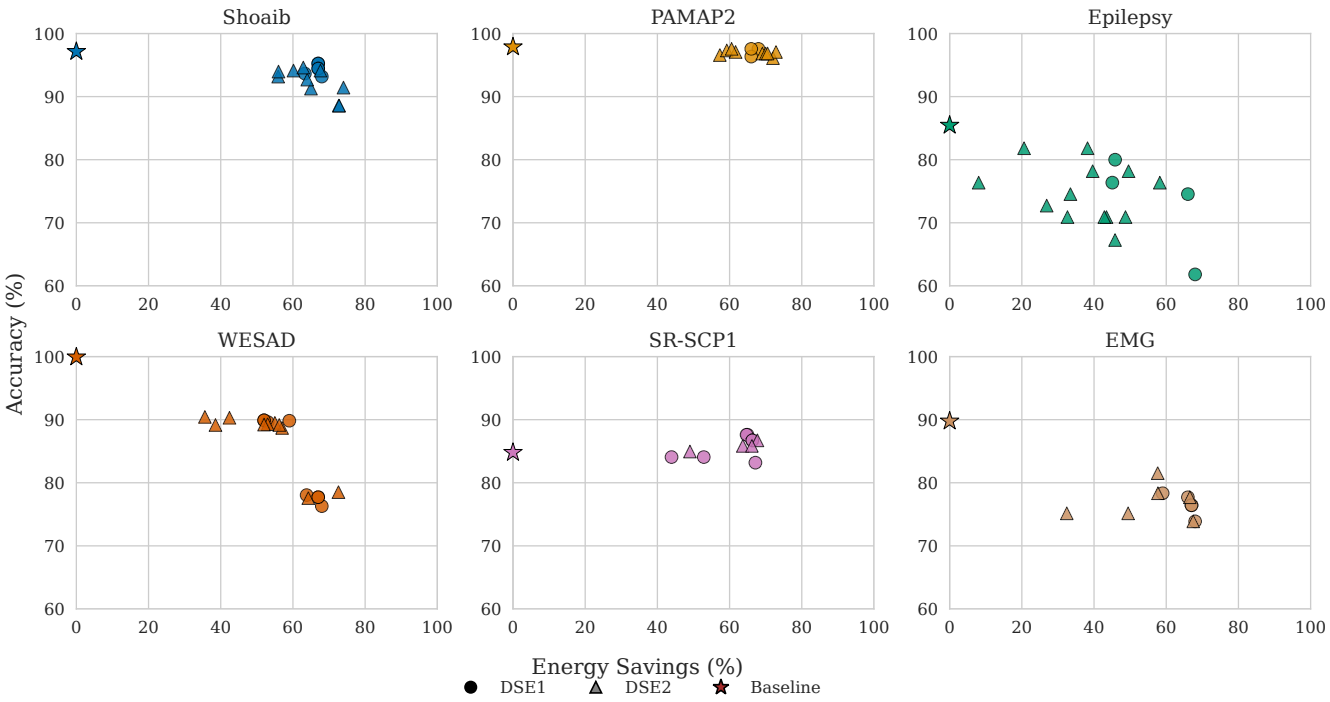

Figure 10: SEE RF-V: Accuracy-Energy trade-off showing Pareto optimal points for two design space exploration methods across six datasets.(DSE1: exhaustive search; DES2: B.O)

(c) **Progressive Evaluation**: Model computes predictions and entropy at tier checkpoints

(d) **Early-Exit Decision**: When entropy $\leq \tau$, CPU sends 'window-done" signal

(e) **Sensor Control**: System evaluates whether to power-gate sensor until next window

We propose non-overlapping (tumbling) windows rather than sliding windows to avoid stale predictions and simplify state management—once a window is processed, its data is discarded entirely.

Power-Gating Decision Logic

After early exit at time t_{exit}, the system must decide whether to power down the sensor. Let:

- $t_{remain} = t_{window} - t_{exit}$ = time remaining in current window
- $t_{overhead} = t_{sleep} + t_{wake}$ = total wake/sleep transition cost

Decision rule:

if $t_{remain} > t_{overhead}$ **then**
$\rightarrow$ Power-gate sensor (net energy savings)
else
$\rightarrow$ Keep sensor active (avoid wasteful transitions)

This ensures energy savings from early model exit are not erased by excessive power-state transitions.

Sensor Reactivation

The system would schedule sensor wake-up to ensure data availability for the next window:

(a) **Sleep Duration**: $t_{sleep_duration} = t_{remain} - t_{wake}$

(b) **Wake Trigger**: Set timer for $t_{window} - t_{wake}$ to initiate reactivation

(c) **Next Window**: Sensor fully operational when $t = t_{window}$

This timing ensures no data loss between windows while maximizing sleep duration.

Real-Time Constraints

Two critical timing requirements must be satisfied:

(a) **Inference Deadline**: Model decision time must be less than sampling interval

$$t_{inference} < \frac{1}{f_s} \tag{3}$$

[TODO: Insert measured RF inference times from our experiments]

(b) **Window Completion**: Total processing must complete within window duration

$$t_{inference} + t_{control} < t_{window} \tag{4}$$

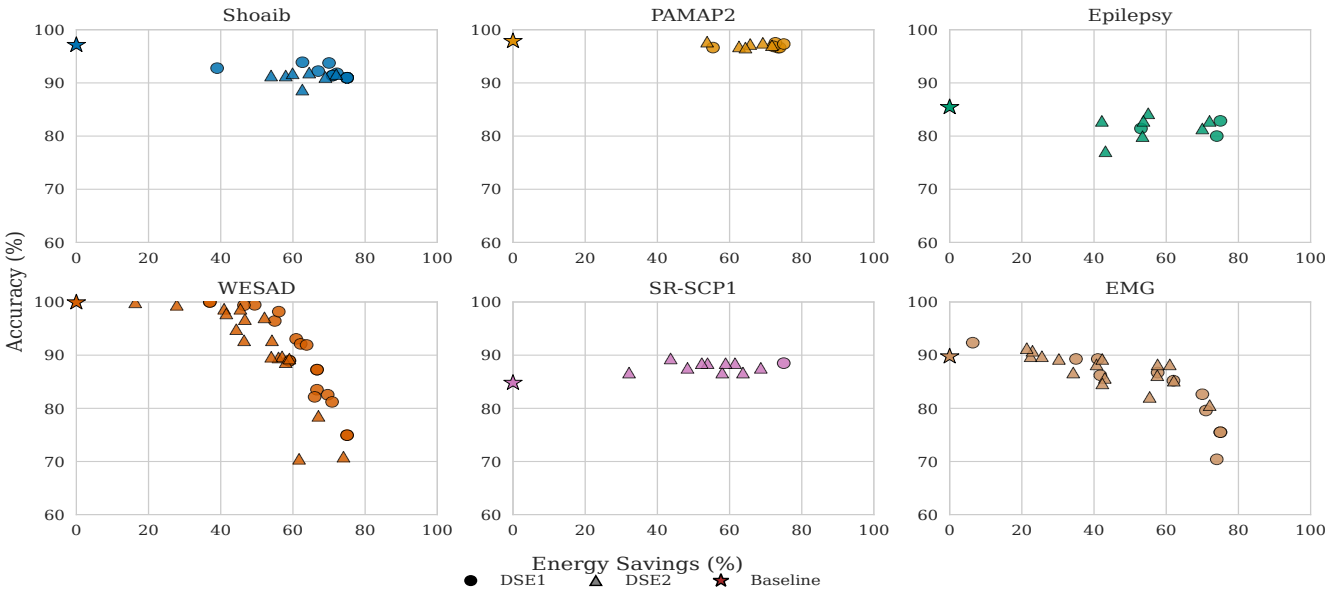

Figure 11: SEE SRF-H: Accuracy-Energy trade-off showing Pareto optimal points for two design space exploration methods across six datasets. (DSE1: exhaustive search; DES2: B.O)

Implementation Requirements

Deployment would require:

- **Sensor Control Interface**: Mechanism to transition sensors between active and low-power states
- **Timing Coordination**: Scheduling for window boundaries and sensor reactivation
- **Data Management**: Buffering to handle continuous sensor streams during inference

Expected Energy Impact

Energy savings would come from two sources:

(a) **Primary**: Reduced computation from early model exit

(b) **Secondary**: Sensor duty-cycling when $t_{remain} > t_{overhead}$

Based on our software evaluation showing early-exit rates of [TODO: insert Table X reference], deployment would enable corresponding duty-cycle reductions. The exact savings depend on platform-specific parameters (sensor power differential, transition latencies) that would be characterized during deployment.

This architecture shows how SEE-RF's early-exit decisions would translate to practical energy savings through intelligent sensor management, without requiring complex sliding-window mechanisms or risking stale predictions. By extending battery life of health monitoring devices, this approach could improve health tracking accessibility for populations where frequent charging infrastructure is limited.

Societal Impact and Sustainability Vision

Our energy-efficient inference architectures address multiple dimensions of AI sustainability and social impact. **A full sensor + compute energy evaluation on NVIDIA Jetson Xavier-LP demonstrates C% total energy savings and $D\times$ latency speed-up across our evaluation datasets. Battery life on a typical E mAh smartwatch extends from F hours to G hours, representing an H% improvement that delays thousands of battery replacements and improves adherence to long-term health monitoring.** *[INSERT: actual energy savings, speedup, battery capacity, and life extension measurements from experiments]*

At the ecosystem level, reducing per-device energy consumption by I% *[INSERT: measured energy reduction]* could save billions of kilowatt-hours annually across the global wearable fleet, while extending device lifespans reduces electronic waste and manufacturing demand. For users in low-resource settings, fewer charging cycles mean better access to continuous health monitoring—a critical factor for managing chronic conditions like diabetes, hypertension, and heart disease where consistent data collection enables early intervention and prevents costly emergency care.

More broadly, this work contributes to a vision of sustainable AI that prioritizes efficiency alongside accuracy. As AI deployment scales globally, the energy efficiency of inference becomes as important as the sophistication of algorithms. By demonstrating that significant energy savings are achievable without sacrificing predictive performance, we provide a concrete pathway toward AI systems that enhance human welfare while respecting planetary boundaries. Our lightweight, partial-data approach

represents a fundamental shift from the current paradigm of data-hungry models toward intelligent systems that achieve more with less—a critical capability for sustainable AI deployment at global scale.

While our approach requires system-level sensor control and may not be suitable for safety-critical applications requiring guaranteed full-window inference, the energy savings demonstrate the potential for sustainable edge AI deployment. All code, datasets, and hyperparameter configurations will be made publicly available for reproducibility. Our finding that random threshold selection outperforms ordered strategies extends prior work on early-exit networks and highlights the importance of allowing each exit point to develop independent confidence characteristics, particularly in ensemble models.

* Add Info about memory overhead * Comparative Study

References

[1] Baumann, F.; Ehlers, A.; Rosenhahn, B.; and Liu, W. 2015. Sequential boosting for learning a random forest classifier. In *2015 IEEE Winter Conference on Applications of Computer Vision*, 442–447. IEEE.

* Add Info about memory overhead * Comparative Study